



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Design Document

SOFTWARE ENGINEERING FOR AUTOMATION — RACE MANAGER

Authors:

- **Campaci Luca** — 10588468
- **Caruso Marco** — 10726368
- **Veronese Francesco** — 10782721

Professors: Prof. Matteo Camilli
Academic Year: 2025–26

Contents

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Scope	1
1.3	Definitions, Acronyms, and Abbreviations	2
1.4	Document Structure	3
2	Architectural Design	4
2.1	Overview: High-level Components and Their Interaction	4
2.2	Component View	6
2.2.1	Component responsibilities	6
2.2.2	Dependencies and boundaries	6
2.3	Deployment View	7
2.3.1	Production topology	7
2.3.2	Development topology	8
2.3.3	Startup, shutdown, and recovery	8
2.4	Runtime View	8
2.4.1	Account access and session renewal	9
2.4.2	Individual and team registration	9
2.4.3	Waiver submission and retrieval	10
2.4.4	Timing subscription and source changes	11
2.4.5	Session commands, pit state, and automatic penalties	11
2.4.6	Official results import	12
2.5	Component Interfaces	13
2.5.1	REST interface families	13
2.5.2	WebSocket protocol	14
2.5.3	Adapter and persistence interfaces	15
2.6	Selected Architectural Styles and Patterns	15
2.6.1	Client–server and modular monolith	15
2.6.2	Model–View–ViewModel	15
2.6.3	Factory and provider adapters	15
2.6.4	Publish–subscribe	15
2.6.5	REST and dependency injection	16
2.6.6	Relational persistence and transaction boundaries	16

2.7	Other Design Decisions	16
2.7.1	Database design	16
2.7.2	Consistency and update strategies among replicas	17
2.7.3	Failures, retries, and resource limits	17
2.7.4	Security and privacy boundaries	18
3	User Interface Design	19
3.1	Navigation and Shared Interaction Rules	19
3.2	Screen Specifications	19
3.3	Mockup Specifications	21
3.4	Interaction and Accessibility Validation	23
4	Requirements Traceability	24
4.1	Performance and Quality Traceability	26
5	Implementation, Integration and Test Plan	28
5.1	Implementation and Unit Testing	28
5.1.1	Priorities and dependencies	28
5.1.2	Unit testing strategy	29
5.2	Integration Testing	29
5.3	System Testing	30
5.4	Acceptance Procedure and Evidence	31

1 | Introduction

1.1. Purpose

The Design Document (DD) explains how Race Manager supports event preparation, live race supervision, and post-race consultation. It provides a shared reference for developers, reviewers, and testers and connects each RASD requirement to the components responsible for fulfilling it.

The design addresses the native iOS application, the Python backend, persistent records, external timing acquisition, and their communication. It specifies failure handling and consistency boundaries as well as successful interactions. The RASD remains the authority for required behavior.

1.2. Scope

The baseline includes account management, role-based access, circuits and events, individual and team registrations, waivers, live standings, kart assignments, pit and stint state, messages, penalties, notifications, and official results.

The system observes timing supplied by external providers. Physical timing equipment, race enforcement, and external payment verification remain outside the software boundary. Recording a cost or `pending_payment` does not implement payments. Adding a member by email does not implement email delivery or invitation acceptance.

The initial deployment uses an iOS client and a single backend host. Android and web clients, mobile push notifications, complete offline operation, payment processing, and advanced telemetry remain future extensions. A browser used to view an exported PDF is not a separate full web client.

1.3. Definitions, Acronyms, and Abbreviations

Term	Meaning
DD / RASD	Design Document / Requirements Analysis and Specification Document
MVVM	Model–View–ViewModel, the client presentation pattern
API / REST	Application Programming Interface / Representational State Transfer
DTO	Data Transfer Object exchanged across a service boundary
ORM	Object–Relational Mapping, provided by SQLAlchemy
JWT	JSON Web Token used for access authentication
HTTPS / WSS	Encrypted HTTP / WebSocket communication
mDNS / Bonjour	Local service discovery mechanism
Source session	Backend acquisition resources associated with a timing URL
Race session	Operational activity within an event, with its own running/paused/stopped state
Snapshot	Latest acquired timing representation for a source
Invalidation	Notification telling a client to retrieve updated authoritative records
Stint	On-track driving interval, excluding paused and pit time
Team leader	Event-specific membership responsibility, not a global role
CSV / PDF	Comma-separated values / Portable Document Format
TBD	To be decided or confirmed

The terms source session and race session are deliberately distinct: selecting a timing URL does not start a race. Requirement identifiers R1–R34, P1–P5 and Q1–Q8 refer to the RASD. Component identifiers C1–C10 and test identifiers T1–T12 are local to this document.

1.4. Document Structure

- **Section 2** describes the architecture, deployment, runtime behavior, interfaces, patterns, and data decisions.
- **Section 3** defines the user-facing flows and screen specifications.
- **Section 4** maps requirements to design elements and verification activities.
- **Section 5** defines implementation priorities, integration order, and tests.

2 | Architectural Design

2.1. Overview: High-level Components and Their Interaction

Race Manager uses a client–server architecture with three logical layers: presentation, application logic, and persistence. These are not three independently deployed server tiers. The SwiftUI application runs on iOS; the FastAPI application and SQLite database reside on the backend host.

The backend is a modular monolith. Domain routers share authentication dependencies, SQLAlchemy models, and database sessions. Playwright acquisition runs in source-specific threads, while WebSocket delivery and periodic background tasks cooperate with the server’s asynchronous runtime.

The principal communication paths are:

1. The iOS application sends REST requests for account, registration, waiver, event, and race-control operations.
2. The backend authenticates the caller, validates permissions and domain rules, and reads or commits database records.
3. An authenticated WebSocket receives source-specific timing snapshots and event-specific update notifications.
4. Source adapters acquire external timing and return a common tabular representation.
5. The client refreshes affected resources after event invalidations and renders timing snapshots for its selected source.

Persistent race decisions are authoritative on the backend. UI state and timing caches are projections. A lost WebSocket notification must not erase a committed registration or penalty; a reconnecting client retrieves current state through REST.

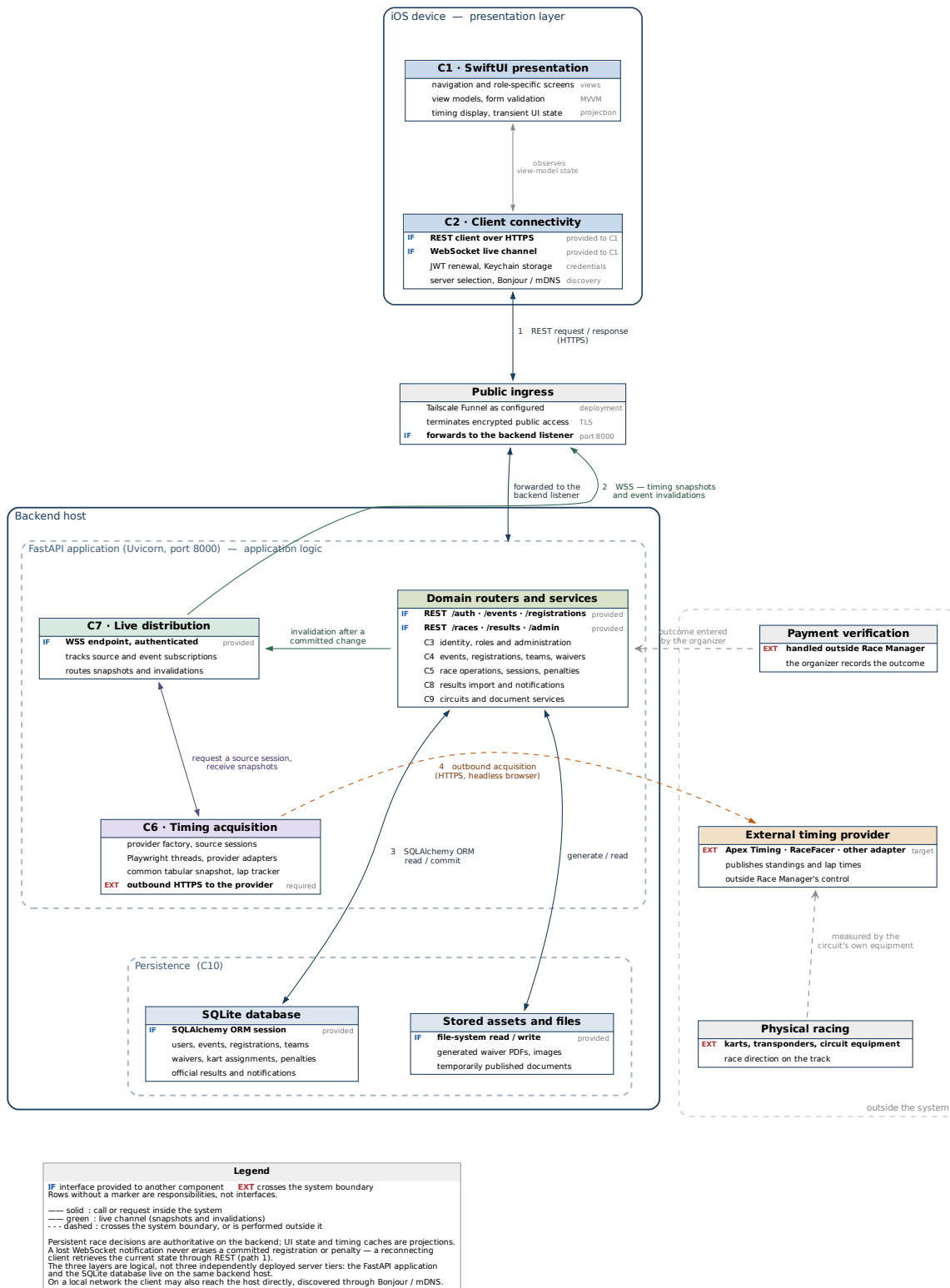


Figure 2.1: High level architecture

2.2. Component View

2.2.1. Component responsibilities

ID	Component	Responsibilities
C1	iOS presentation and application state	Navigation, role-specific screens, form validation, view models, timing display, and transient UI state
C2	Client connectivity and credentials	REST requests, token renewal, Keychain access, server selection, local discovery, and live connection coordination
C3	Identity and administration	Account lifecycle, password verification, current-role authorization, user search, and role changes
C4	Events and participation	Event lifecycle, registration routing, teams, organizer decisions, and waiver submission/status
C5	Race operations	Kart assignments, session controls, pit timers, messages, penalties, and automatic stint assessment
C6	Timing acquisition	Provider selection, source resource lifecycle, common snapshots, and observed lap extraction
C7	Live distribution	Authenticate WebSockets, track source/event subscriptions, and route snapshots and invalidations
C8	Results and notifications	Official CSV import, classifications, lap summaries, recipient notifications, and event reminders
C9	Circuits and document services	Circuit/source configuration, waiver PDF generation, administrative waiver access, and temporary PDF publication
C10	Persistence and host lifecycle	Database connections/models, startup initialization, stored assets, task startup/shutdown, and Bonjour publication

These are logical responsibilities, not claims that each is an independently deployable service. In the current code, several business operations live directly in routers. Extracting shared policies into service functions is a target refinement where it reduces duplicated validation; a separate microservice architecture is not required.

2.2.2. Dependencies and boundaries

C1 invokes C2 and observes view-model state. C2 communicates with backend interfaces rather than reading the database. C3 supplies identity and role dependencies to C4, C5, C8, and C9. Those domain components use C10 for persistent data and C7 for live invalidations after successful changes.

C7 requests source sessions from C6 and receives snapshots for delivery. C6 knows provider

formats; registration and waiver components do not. C8 reads observed lap records without treating them as an official classification. C9 generates waiver PDFs from authorized records rather than granting access based on a filename supplied by the client.

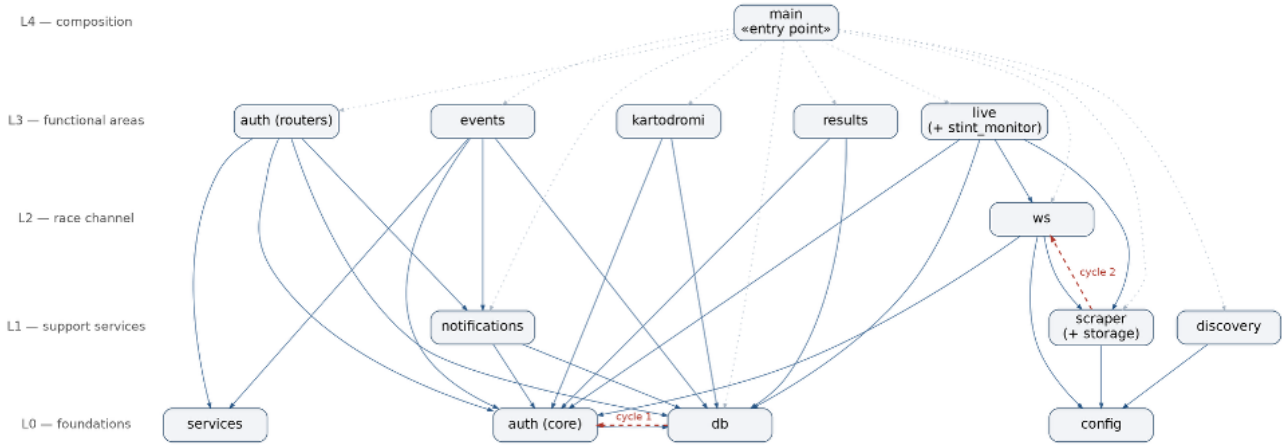


Figure 2.2: Component view

2.3. Deployment View

2.3.1. Production topology

Node	Deployed artifacts	Communication and responsibility
iOS device	SwiftUI application, Keychain entries, transient view models	HTTPS requests and WSS connection to the configured remote endpoint
Public ingress	Tailscale Funnel as configured for the project	Terminates public encrypted access and forwards to the backend listener
Backend host	Uvicorn/FastAPI, Python modules, Playwright/browser resources, periodic tasks	Application execution and outbound access to allowed timing providers
Backend storage	SQLite database and approved asset directories	Persistent domain records and files; transient timing snapshots are disposable
Timing provider	Apex Timing, RaceFacer, or another supported adapter target	External timing source outside Race Manager's control

The repository configures Uvicorn on port 8000. The production hostname is an environment-specific value in the client configuration and is not an architectural constant. The chosen

host must remain running during service; no high-availability cluster or failover guarantee is established. The choice of Tailscale Funnel was made to test the connection over internet without having to buy a domain.

2.3.2. Development topology

Development mode supports a selected local host and port discovered with Bonjour using `_karttiming._tcp.local..` The client may use local HTTP in this explicitly selected mode. The simulator provides controlled timing fixtures without depending on a public race.

Development and production endpoint selection must be visible to maintainers. Changing discovery results must not grant privileges or silently change the production transport policy.

2.3.3. Startup, shutdown, and recovery

Existing startup initializes the database, closes already expired events, clears saved timing data, starts Bonjour, and schedules reminder, stint-monitor, and event-expiration tasks. Shutdown cancels these tasks, stops source sessions, stops discovery, and clears disposable timing files.

Database records survive normal restart. WebSocket connections and source sessions do not; clients must authenticate, resubscribe, and refetch event state.

In case of an unpredicted shutdown a recovery policy is yet to be defined.

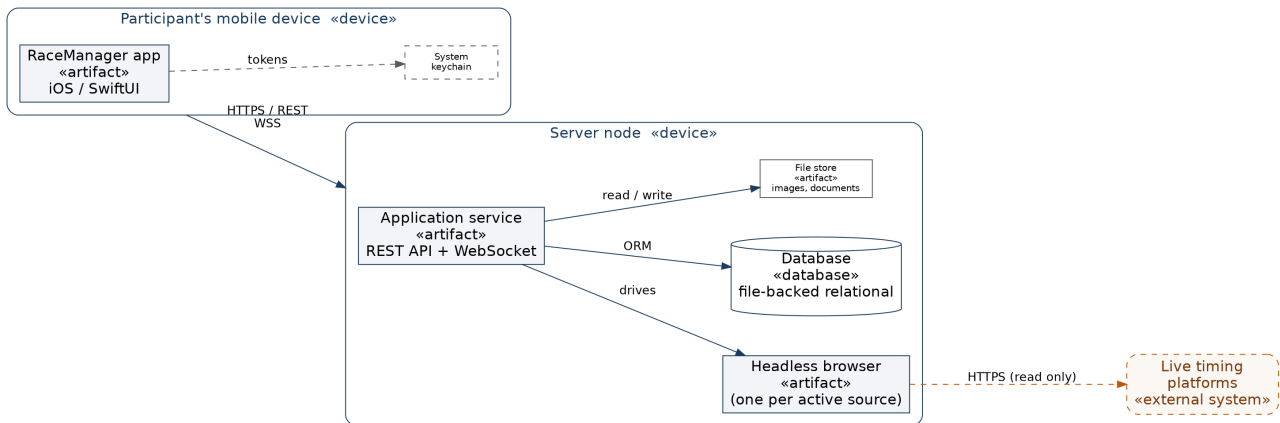


Figure 2.3: Deployment view

2.4. Runtime View

The sequences below specify transaction boundaries and recovery expectations. Where described as target behavior, they require implementation review and validation before being claimed as supported.

2.4.1. Account access and session renewal

1. The login screen sends credentials to C3 through C2.
2. C3 verifies the password hash and returns access/refresh credentials on success.
3. C2 stores credentials through the client credential facilities and loads the current profile.
4. Protected requests resolve the current database account and role; a role embedded in an old token is not the sole authorization source.
5. When access renewal is required, C2 uses the refresh endpoint and retries only under the operation's retry policy. Failed renewal returns the user to authentication.
6. Logout revokes the refresh token and clears client credentials. Revoking a refresh token is not a claim that every already-issued access JWT is immediately revoked.

The WebSocket validates identity before acceptance and revalidates while receiving commands or after an idle timeout. Deleted accounts and expired access tokens must not retain protected live access indefinitely.

2.4.2. Individual and team registration

1. C1 retrieves event details and the participant's existing registration and waiver status.
2. C4 checks authenticated identity, event existence, deadline, format, duplicate participation, and applicable size/capacity rules.
3. An admissible request becomes **pending_payment**; a late, full, or team-awaiting individual request becomes **waitlist** under the RASD policy.
4. Team creation groups member registrations by an event-local team identifier and marks one leader. An unlinked email is not treated as an existing authenticated account.
5. The target transaction commits the complete operation and its recipient notifications together. Invalid member data or a conflict rolls back the operation rather than leaving half a team.
6. The response supplies authoritative status. The client refreshes after a timeout before deciding whether to retry, since the original request may have committed.

Organizer confirmation and waiting-list admission use the same eligibility rules and event boundaries. Team-wide operations must update the intended group consistently. Participant cancellation of a confirmed entry and non-leader team departure are distinct workflows.

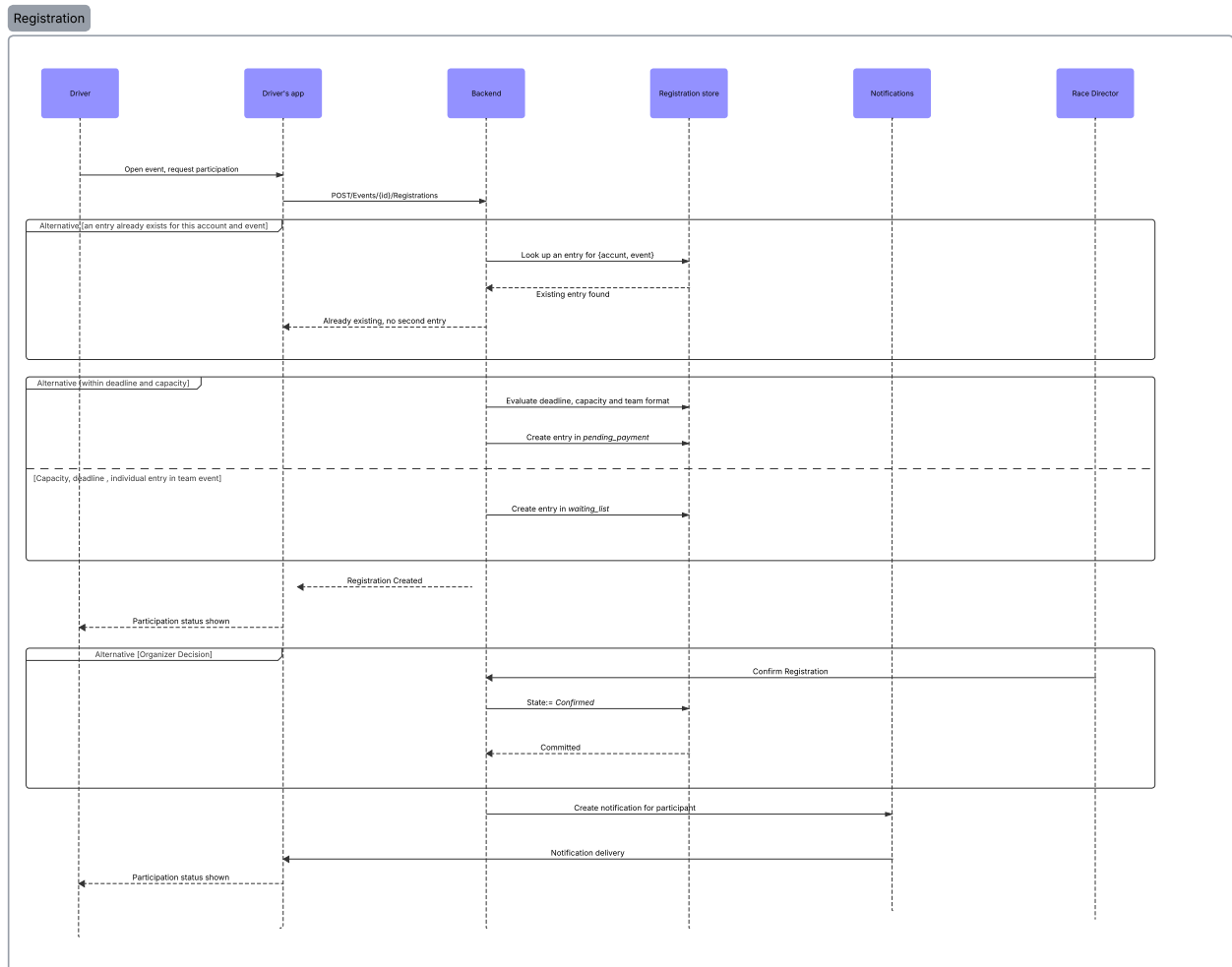


Figure 2.4: Registration sequence

2.4.3. Waiver submission and retrieval

1. The participant retrieves the applicable waiver and requests a preview.
2. The client collects the required personal details and drawn signature.
3. C4 validates and stores a **SignedRelease** linked to the authenticated user and event.
4. Only a successful response changes the visible signed status.
5. An administrator uses the protected waiver endpoint; C9 reads the record and generates the PDF.
6. Removal/invalidation is reflected by retrieving current waiver status.

The database stores signature data and participant details; a generated PDF is a representation of those records. Signed status does not automatically confirm participation.

2.4.4. Timing subscription and source changes

1. C2 opens `/ws` with a valid access token. Initially no source is selected.
2. The client sends `set_url`; C7 checks command shape, role, URL policy, source-update state, and session limits.
3. C7 moves only that socket's subscription. C6 reuses a session for an existing URL or creates one for a new URL.
4. C7 acknowledges `url_changed` and may immediately send the session's cached snapshot.
5. The adapter acquires a table; C6 wraps it with source identity and acquisition time, stores disposable timing data, and invokes lap tracking.
6. C7 broadcasts only to sockets mapped to that source.
7. When the last subscriber leaves, the session stops after the configured grace period unless another subscriber returns.

Current configuration uses a 3-second polling interval, a 15-second idle grace period, and at most 5 concurrent source sessions. These are settings, not measured throughput guarantees. Acquisition may take additional time.

2.4.5. Session commands, pit state, and automatic penalties

Event lifecycle (`scheduled`, `started`, `finished`) and race-session state (`not_started`, `running`, `stopped`) are stored separately. The following table expresses target transition semantics from the RASD.

Action	Preconditions	Result and timer effect
Explicit session start	Authorized official; event eligible for a session	Session becomes running; initializes the new session's stint state
Checkered and red flag	Active running session	Freeze timers and stop
Pit entry	Valid event kart and permitted operation	Preserve elapsed time and stop accumulating for that kart
Pit exit	Kart in pit	Begin a new stint; start accumulation only if the session is running; reset automatic-penalty eligibility
Event closure	Authorized operation or provisional expiration rule	Close the event and reconcile operational state; do not publish results implicitly

The existing backend monitor scans approximately once per second. It uses a SQLite write transaction, conditionally claims `stint_penalty_assessed`, and inserts the automatic penalty

in the same transaction. The marker persists even if the penalty is deleted. A new stint rearms assessment. Thus the intended guarantee is at most one automatic assessment per stint, not once per monitor tick or connected client.

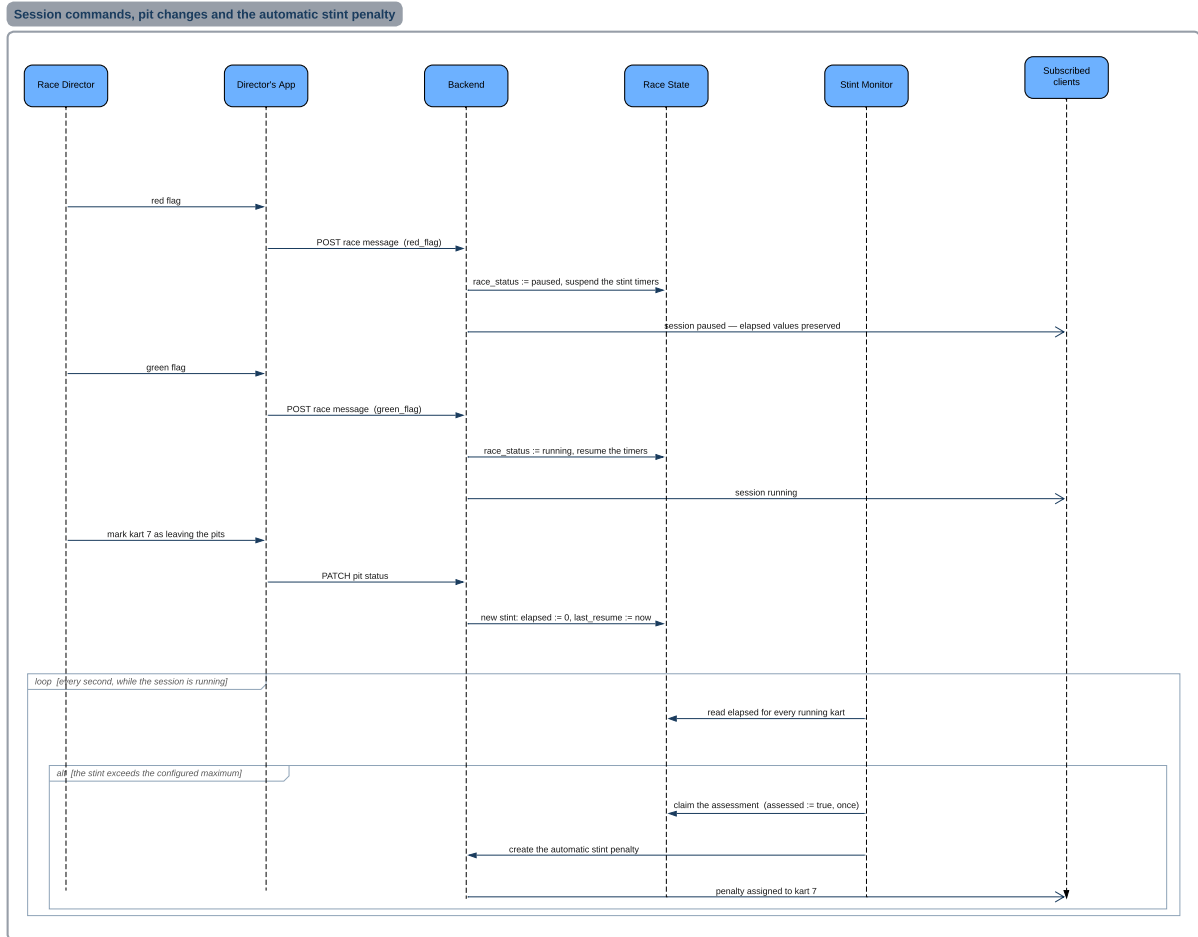


Figure 2.5: Session and stint sequence

2.4.6. Official results import

1. An administrator submits a CSV for an existing event and result type.
2. C8 parses rows, validates usable values, and attempts participant/team association.
3. Parsed valid rows are staged before replacement. If none are valid, the previous classification remains unchanged and an error is returned.
4. When valid rows exist, the accepted replacement is committed and row-level problems are reported.
5. Clients retrieve official results and distinguish unassociated entries and unavailable lap summaries.

The current parser accepts provider-oriented column names such as `Posizione`, `Kart`, and `Pilota/Squadra`. Live estimates and observed laps do not automatically become official results.

2.5. Component Interfaces

2.5.1. REST interface families

Interface	Access and contract
Authentication	Public registration/login; renewal and logout validate their credential inputs
Profile	Current authenticated account; no self-assigned global role
Events	Writes require director or administrator
Participation	Own participation; target minimum role is <code>user</code>
Teams	Leader/member/official checks depend on operation; event scope is mandatory
Organizer decisions	Director or administrator; atomic eligible transitions
Waiver	Participant-specific submission and status
Waiver administration	Administrator access to event-bound records
Event/session status	Authorized operational transition
Live operations	Authenticated permitted reads; director/admin writes
Personal race state	Resolve the caller's event entry and applicable race state
Official results	Administrator-only import; permitted classification consultation
History and laps	Caller-bound history and permitted event summaries
Notifications	Recipient-specific access
User administration	Administrator only
Circuit configuration	Permitted consultation; administrator maintenance
Temporary documents	Authenticated upload; existing download uses an expiring capability URL, discussed below

Protected REST calls use bearer access credentials. JSON is used for ordinary structured

operations; CSV uploads use multipart form data and PDF responses use the appropriate binary media type. Successful mutations return only after their transaction succeeds. HTTP 401/403 distinguish authentication and authorization failures; missing resources, malformed inputs, and domain conflicts must produce recoverable client feedback. Exact existing status codes remain defined by each route; a uniform conflict response is a target consistency improvement.

2.5.2. WebSocket protocol

The existing connection is `/ws?token=<access_token>`. Token-bearing URLs must not be retained in ordinary access logs or diagnostics. Authentication failure closes the connection with code 4401 in the token verifier.

Direction	Message	Semantics
Client → server	<code>{"command": "set_url", "url": "https://simulator"}</code>	Select one allowed source for this socket
Client → server	<code>{"command": "get_status"}</code>	Ask for the socket's current source/session status
Client → server	<code>{"command": "subscribe_event", "event_id": 42}</code>	Select an event update context; positive integer required
Server → client	<code>{"type": "url_changed", "url": "https://simulator"}</code>	Acknowledge source selection
Server → client	<code>status</code> with <code>scraping</code> , <code>url</code> , and <code>role</code>	Current subscription status; not a freshness guarantee
Server → client	<code>timing_update</code>	Snapshot with <code>url</code> , <code>updated_at</code> , <code>headers</code> , and <code>rows</code>
Server → client	<code>event_update</code>	Invalidate event resources and retrieve current state
Server → client	<code>error</code> with <code>message</code>	Invalid command, disallowed source, unavailable capacity, or another operation error

Some existing event updates include `event_id`, `change`, `karts_changed`, and `request_id`; others contain only `type`. Clients must handle this baseline variation.

Source and event subscriptions use separate maps; selecting a source does not prove an event association. The existing event-subscription branch validates the identifier's shape but does not establish a complete event-access policy. Protected event payloads require explicit authorization and server-side recipient filtering. Private notification bodies and waiver data must not be broadcast merely because a client names an event.

2.5.3. Adapter and persistence interfaces

`BaseScraper` defines `setup(url)`, `scrape()`, and `teardown()`. `scrape()` returns `headers: list[str]` and tabular rows. The factory chooses the provider implementation; `ScraperSession` owns polling and resource lifetime. The current common contract is tabular, not a fully typed kart DTO. Semantic normalization must preserve kart identity and distinguish unavailable data from zero values; a future typed representation requires coordinated client/server changes.

`get_db()` supplies a request-local SQLAlchemy session. Background jobs open their own sessions. Sessions must not be shared between an acquisition thread and unrelated requests. Foreign-key enforcement is enabled when connecting. Database integrity exceptions must be rolled back and translated into domain outcomes.

Current timing timestamps use a local, timezone-naive ISO string in the acquisition loop.

2.6. Selected Architectural Styles and Patterns

2.6.1. Client–server and modular monolith

One backend owns domain policies and persistent state. Module boundaries keep account, registration, timing, and race-control concerns understandable without introducing distributed transactions. This simplifies the initial deployment but makes the backend host a single availability dependency.

2.6.2. Model–View–ViewModel

SwiftUI views present state and collect user actions. View models coordinate feature loading and updates; models represent domain/API data. Shared authentication and environment objects support navigation and connectivity. Server rules remain authoritative even when the UI disables an action.

2.6.3. Factory and provider adapters

The scraper factory selects an adapter implementing `BaseScraper`. Provider-specific page structure remains outside event and waiver logic. A new provider must satisfy normalization fixtures and lifecycle tests before being enabled in the allowed-source configuration.

2.6.4. Publish–subscribe

C7 routes timing snapshots by source and domain invalidations by event. Multiple subscribers share acquisition work. This is an in-process mechanism with best-effort delivery, not a durable message broker. REST refetch supplies recovery for missed invalidations.

2.6.5. REST and dependency injection

Resource-oriented endpoints expose domain operations to the native client and potential future clients. FastAPI dependencies centralize identity, role checks, and database-session provisioning. Dependency injection does not replace resource-level ownership checks inside operations.

2.6.6. Relational persistence and transaction boundaries

SQLAlchemy maps persistent entities to SQLite. Transactions group related changes, while database uniqueness protects selected invariants even under concurrent requests. The design does not use a data warehouse or replicated operational database. Introducing either would require a demonstrated workload need.

2.7. Other Design Decisions

2.7.1. Database design

Entity/table	Main references and data	Integrity responsibility
User / users	Identity, profile, password hash, global role	Unique username/email; permitted roles
RefreshToken / refresh_tokens	User, token hash, expiration, revoked flag	Credential renewal and revocation
Event / events	Schedule, limits, waiver text, lifecycle/session states	Valid limits and permitted transitions
Kartodromo / kartodromi	Circuit metadata and unique source URL	Allowed provider configuration
EventRegistration / event_registrations	Event, optional user, email, team identifier, leader, status	Unique linked user/event; team rules in application logic
SignedRelease / signed_releases	Event, user, submitted details, signature, signing time	Unique event/user waiver record
LiveKartAssignment / live_kart_assignments	Event, entry/team identifier, kart, pit state, elapsed interval, assessed marker	Unique event/kart; eligibility and assignment validation
PenaltyType / penalty_types	Code, action, default seconds, warning threshold, automatic consequence	Valid configuration and non-negative time values
RacePenalty / race_penalties	Event, kart, type, time value, note	Event-local race decision

RaceMessage / race_messages	Event, optional target kart, type, text	Global control commands cannot target one kart
EventResult / event_results	Event, optional user/team, position, laps, best lap, official flag	Validated replacement and explicit association status
LapTime / lap_times	Event, kart, lap number, milliseconds	Correct source/event association and duplicate handling
Notification / notifications	Recipient, optional event, message, read flag	Recipient-only read/delete
EventReminder Delivery / event_reminder_deliveries	Composite user/event key and sent time	Persistent reminder deduplication
KartodromoResult / kartodromo_results	User, circuit, best lap, date	Personal circuit result, separate from official event classification

There is no separate persistent Team table: registrations sharing a team identifier form the current group. Likewise, `Event.location` is not an explicit foreign key to `Kartodromo`. These implementation facts matter when validating event/source associations and evolving the schema.

Observed laps depend on source acquisition and association. Acquisition can stop without subscribers, so continuous complete lap history is not guaranteed by the present architecture. Official result publication remains a distinct workflow.

2.7.2. Consistency and update strategies among replicas

The baseline has no database replicas. Client view models and source snapshot files are caches, not independent writable authorities. A successful REST mutation establishes durable state; event invalidations trigger refresh. On reconnect or foreground return, clients refetch relevant event, registration, and live-operation records.

Capacity checks and registration insertions must form one serialized decision under concurrency. Existing unique keys cover duplicate linked accounts and kart numbers, but not total event capacity.

2.7.3. Failures, retries, and resource limits

Read operations can generally be retried after connectivity recovery. The UI should show that the outcome is being checked rather than invent a failure or success.

CSV, signature, and image limits must be documented and enforced consistently. The existing temporary PDF service caps uploads at 10 MB and expires links after one hour. Other limits remain subject to measurement and interface review. A resource-limit response must preserve the user's existing subscription and committed data.

2.7.4. Security and privacy boundaries

Roles follow `viewer < user < race_director < admin`. The server checks current roles and relevant ownership/membership. Directors currently operate across events. Official result import remains administrator-only.

3 | User Interface Design

3.1. Navigation and Shared Interaction Rules

The authenticated application provides role-appropriate entry points for home, events, timing/live operations, analysis, and profile/administration. Team responsibilities derive from the selected event. They do not add a global Team Manager role.

The intended navigation for *user* event subscription is:

Sign in → Home → Events → Event details → Registration / Waiver

If an event is started, the registered *user* navigation is:

Home → Go to live → Live timing / Team view / Messages

If an event is ended, the registered *user* navigation is:

Home → Analysis

An in-app notification can open the associated event after verifying that the event remains available. Logout clears account-specific state and returns to authentication.

Every data screen distinguishes loading, content, empty, error, and unavailable/stale states where applicable. Important statuses use text as well as color. Destructive actions identify the affected event or entry. Client validation provides early feedback while displaying server rejection if authoritative rules have changed.

3.2. Screen Specifications

Screen	Main information and actions	Required feedback and access behavior
Sign in and account creation	Credentials, account details, submit	Field errors, duplicate account, failed credentials, in-progress state
Home and notifications	Upcoming/relevant events, recipient messages, read/delete	Empty inbox, unavailable event link, recipient-only operations

Event list and details	Date, location, format, limits, cost, deadline, lifecycle	Registration, waiver, and event status shown separately
Individual registration	Participation request and applicable details	Explicit pending/waitlist outcome; eligible cancellation only
Team registration/edit	Name, leader, member identifiers, size limits	Duplicate identity, pending account link, unauthorized edit, leave-team outcome
Waiver preview/sign	Conditions, personal details, signature pad, preview	Missing data/signature, upload failure, confirmed stored status
Live standings	Selected source, positions, kart identifiers, laps, gaps, freshness	Connecting, stale feed, disconnected service, unknown values
Participant/team live view	Assigned kart, stint elapsed, pit state, penalty total, applicable messages	No assignment, paused/stopped session, stale data
Director event management	Event form, entry list, waiting list, confirmation controls	Invalid dates/limits, full event, rejected transition, team-wide action result
Director race control	Session name/state, kart assignment, pit controls, messages, penalties	Global versus targeted scope, action confirmation, conflict and pending-operation state
Results and analysis	Official classification, personal history, available lap summaries	No publication, unassociated participant, unavailable laps, import warnings
Administrator tools	Account search, roles, circuits, waiver administration, CSV import	Permission errors, invalid source, PDF failure, partial/failed import

3.3. Mockup Specifications

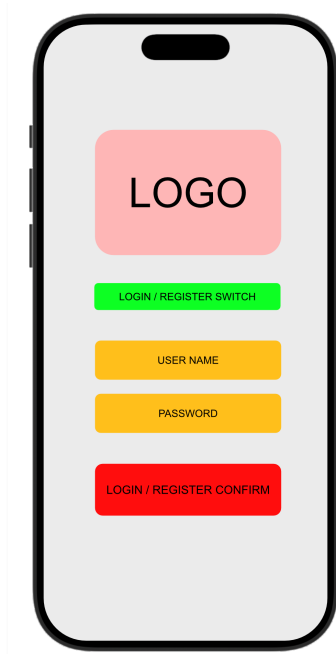


Figure 3.1: Login screen mockup

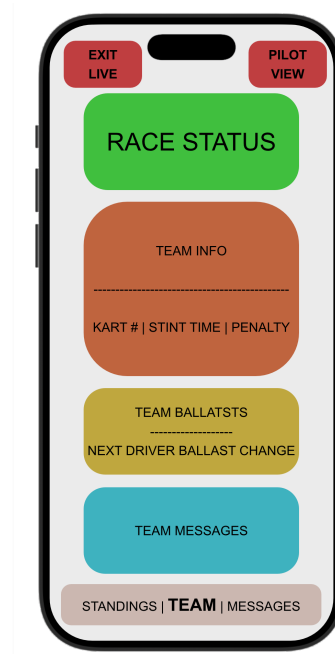


Figure 3.2: Participant screen mockup

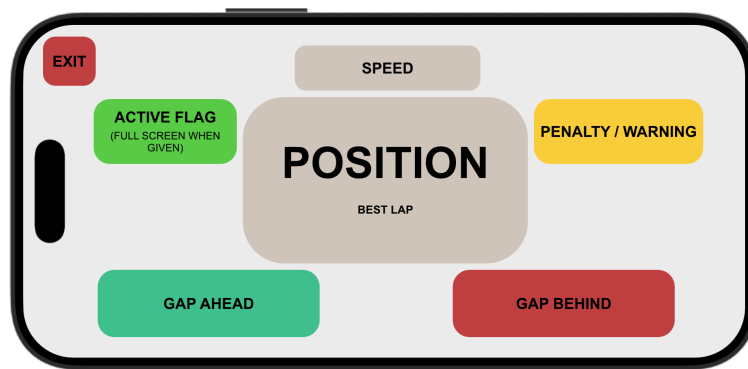


Figure 3.3: Pilot view mockup

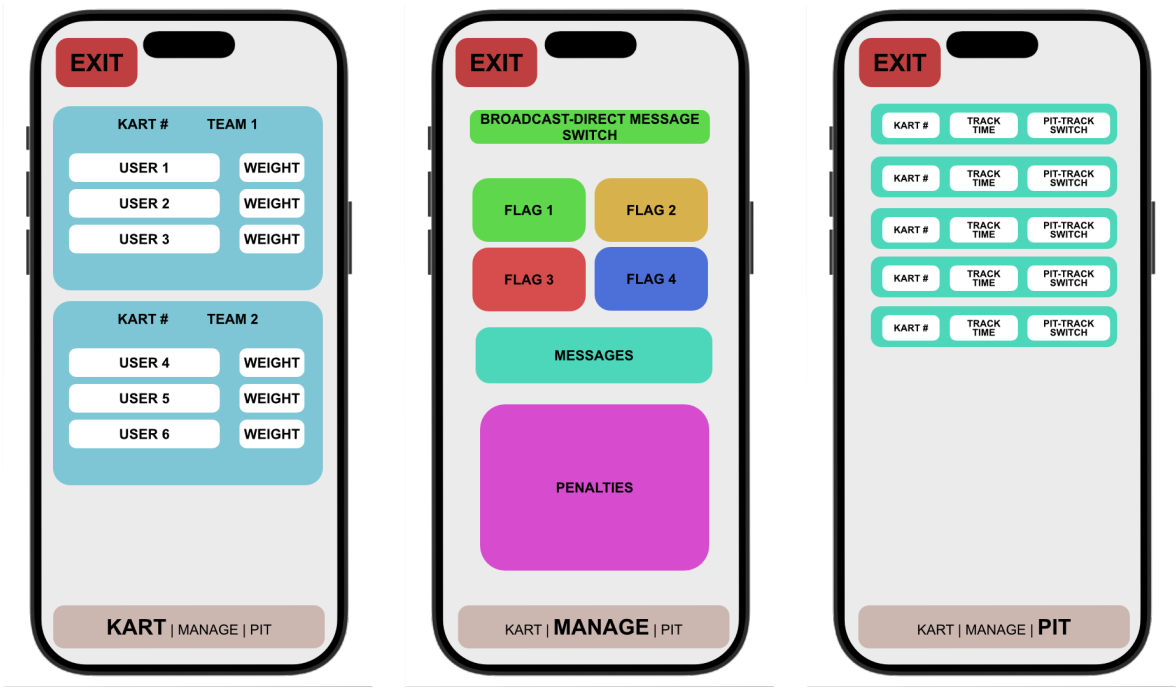


Figure 3.4: Admin event management mockup

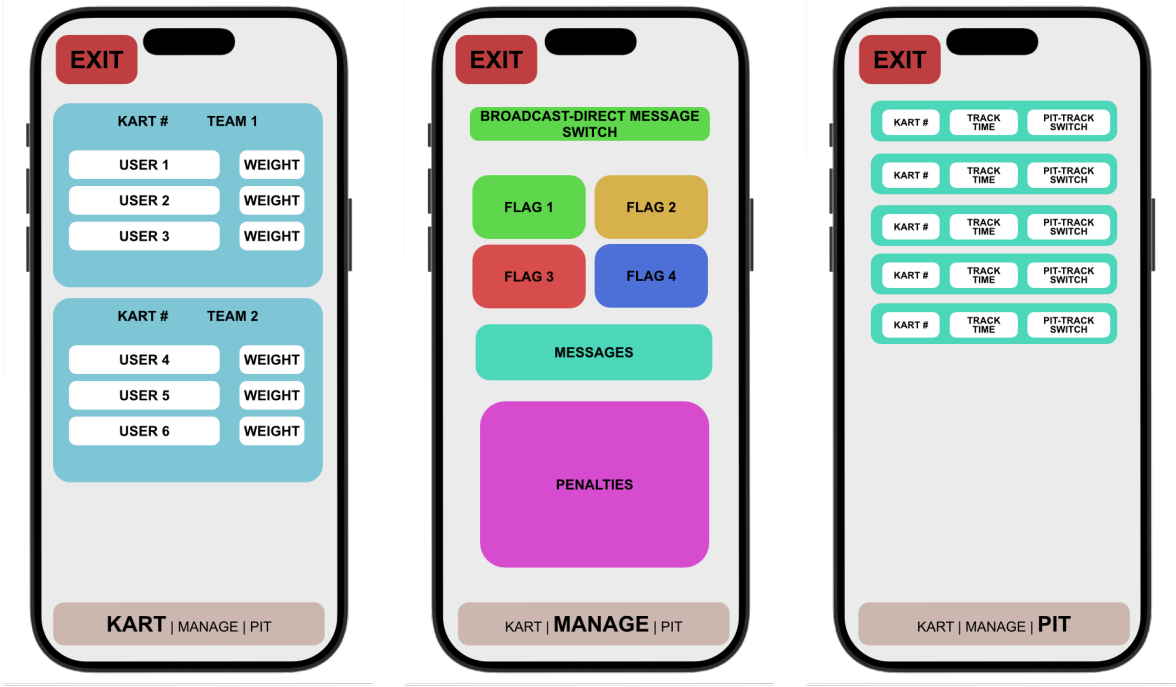


Figure 3.5: Race control mockup

3.4. Interaction and Accessibility Validation

Review the screens on supported iPhone sizes and with larger text. Standings and control labels must remain readable in the intended trackside environment. Verify contrast, clear touch targets, screen-reader labels for controls, and status information that does not depend only on color.

During a pending race-control request, prevent accidental duplicate submission. If connectivity is lost after submission, retrieve current state before offering a retry. Automatic timers and penalty decisions remain backend-owned when the app is backgrounded.

4 | Requirements Traceability

The matrix maps every functional requirement in RASD to a responsible design element and a planned verification group. Coverage indicates design responsibility, not proven implementation. Test groups are defined in the following section.

Requirement	Responsible components	Design mechanism	Verification
R1	C1, C2, C3, C10	Validated non-privileged registration; unique account identities	T1
R2	C2, C3, C10	Credential verification, renewal, refresh revocation	T1
R3	C3, C4, C5, C7, C8, C9	Current-role resolution and resource ownership; live revalidation	T1, T12
R4	C1, C3	Own profile fields separated from role administration	T1
R5	C1, C4	Event DTOs and event list/detail views	T2
R6	C1, C4, C9	Separate registration and waiver status queries	T2, T4
R7	C3, C4, C10	Authorized event mutations and dependent-record consistency	T2
R8	C4, C10	Eligibility checks and linked user/event uniqueness	T2
R9	C4	Deadline, format, and capacity routing to waitlist/pending	T2, T3
R10	C1, C4	Explicit outcome and eligible self-cancellation	T2
R11	C4, C10	Event-local team grouping, leader, identity and size checks	T3
R12	C3, C4	Leader edit checks and separate member departure	T3, T12

R13	C4	Authorized entry/team admission and confirmation transitions	T2, T3
R14	C1, C4, C10	Atomic validation and distinct membership/status projections	T2, T3
R15	C1, C4, C9	Waiver text, preview, data collection, signature capture	T4
R16	C4, C10	Unique event/user storage and acknowledgment-based signed status	T4
R17	C3, C4, C9	Protected PDF retrieval and refreshed invalidation status	T4, T12
R18	C2, C6, C7	Authenticated socket-local source selection	T5
R19	C6	Adapter table contract and semantic normalization	T5
R20	C7	Separate source/event subscription maps and scoped delivery	T5, T12
R21	C1, C2, C6, C7	Freshness state, reconnect, resubscribe, and new snapshot	T5, T10
R22	C5, C10	Authorized assignments and unique event/kart key	T6
R23	C4, C5	Separate lifecycle/session state and explicit new start	T6
R24	C5	Persisted elapsed intervals and pit/pause transitions	T6
R25	C1, C5	Caller-relevant kart, stint, penalty, and message projection	T6, T12
R26	C5	Message target validation and global session-command scope	T6
R27	C5, C10	Valid penalty types/values and configured warning consequences	T7
R28	C5, C10	Backend monitor with transactional assessment marker	T7
R29	C4, C7	Startup/periodic expiration and event invalidation	T9
R30	C3, C8, C10	Admin import, staged validation, safe replacement	T8

R31	C1, C6, C8	Official/history/lap queries with explicit missing associations	T8
R32	C4, C8, C10	Persistent recipient notifications and reminder deduplication	T9, T12
R33	C3	Administrator account search and supported role assignment	T1, T12
R34	C3, C9	Administrative circuit/source and waiver configuration	T4, T5, T12

4.1. Performance and Quality Traceability

Requirement	Design response	Verification
P1	Bound ordinary API work; review database contention and avoid blocking the event loop	T11: 95% within 2 seconds under the RASD reference load
P2	Share acquisition and scope broadcasts; isolate slow consumers	T11: 95% of accepted snapshots delivered within 1 second
P3	Client freshness and connection tracking	T10: stale/disconnected indication within 10 seconds
P4	Periodic backend scan with transactional assessment	T7, T11: continuous violation assessed within 2 seconds
P5	Validate and replace CSV rows as one bounded import operation	T8, T11: up to 200 rows within 5 seconds after upload
Q1	Source-specific sessions and recoverable provider errors	T5, T10: one provider outage while another and REST remain usable
Q2	Durable transactions and explicit post-restart refresh	T10: restart and failed-transaction scenarios
Q3	Encrypted production ingress, hashes, current-role checks, controlled provisioning	T1, T12: direct unauthorized requests and deployment review

Q4	Ownership checks and private storage/document boundaries	T4, T9, T12: cross-account and direct-file attempts
Q5	Textual status, actionable errors, role-appropriate flows	T10: device walkthrough and accessibility review
Q6	BaseScraper adapter boundary	T5: add a fixture adapter without changing participation logic
Q7	Documented REST/WebSocket boundaries	T5, T8: contract fixtures independent of SwiftUI views
Q8	Explicit units, UTC target contract, event/source identity	T2, T5, T6, T10: deadline/timezone and concurrent-event checks

5 | Implementation, Integration and Test Plan

This plan applies to completing and validating an existing prototype. The existing test files consist of ad hoc, synthetically generated data for test purposes.

5.1. Implementation and Unit Testing

5.1.1. Priorities and dependencies

Increment	Work and deliverable	Exit condition
I1	Resolve baseline role rules, secure storage/provisioning, confirm data invariants	Approved permission matrix; no unintended private static access; isolated test database
I2	Validate account/session/profile behavior	T1 unit and interface cases pass
I3	Reconcile event, registration, and team policy including concurrent admission	T2–T3 pass under approved capacity/departure rules
I4	Complete waiver lifecycle and document access decisions	T4 passes with approved revision/sharing policy
I5	Validate provider contracts, source isolation, and live recovery	T5 and related T10 cases pass with deterministic fixtures
I6	Validate race transitions, pit timing, penalties, and downtime recovery	T6–T7 pass with controlled time and no mobile dependency
I7	Validate results, notifications, reminders, and lifecycle closure	T8–T9 pass; import failure preserves data
I8	Complete device, load, security, and restart acceptance	T10–T12 pass or limitations are explicitly accepted

I2 depends on I1. Participation and timing can then progress independently; waivers depend on identity and events. Race operations depend on event identity and live update contracts.

Results need stable event/participant associations. Final system validation depends on the integrated increments.

5.1.2. Unit testing strategy

Use temporary databases and fictional accounts. Do not execute integration fixtures against the repository's operational database or reuse real signatures. Inject or control time for deadline, stint, expiration, and reminder checks. Replace external providers with fixtures for deterministic parsing and error cases.

Unit boundaries include role comparisons, identity normalization, state transitions, timer accumulation, warning thresholds, CSV row parsing, lap-value conversion, and provider selection. Verify observable invariants and error outcomes rather than duplicating implementation statements in tests.

Backend test modules include access integrity, registration integrity, notification integrity, WebSocket routing/management, event expiration/reminders, lap statistics, penalty configuration, stint monitoring, circuit updates, message scope, and temporary PDFs. Coverage of authentication renewal, result replacement, SwiftUI behavior, and full deployment conditions must be assessed and extended where needed.

5.2. Integration Testing

Integrate incrementally around persistent state and service boundaries:

1. **Identity + persistence:** create an account, authenticate, refresh, update profile, and reject direct privilege changes.
2. **Events + registration + notifications:** create an event, register entries, perform team-wide decisions, and check persisted recipient notifications and rollback.
3. **Waivers + document generation:** preview, submit, retrieve as an administrator, remove, and refresh participant state.
4. **Adapters + source sessions + WebSockets:** feed deterministic provider tables, share a source, switch one client, and verify isolation and cleanup.
5. **Race operations + persistence + event updates:** execute session/pit changes and automatic assessment while clients retrieve canonical state.
6. **Results + associations + analysis:** import fixtures with known and unknown participants, validate replacement, and display official status.
7. **Full client + backend:** exercise reconnect, token expiry, event switching, foreground return, and notification navigation.

5.3. System Testing

Group	Scenario and expected observable outcome
T1 - Identity	Duplicate account rejected; profile cannot change role; expired/deleted identity denied; refresh/logout behave as specified
T2 - Events and individual entries	Deadline/capacity routing, duplicate submission, confirmed cancellation restriction, concurrent admission, dependent-record consistency
T3 - Teams	Leader-only editing, valid size, duplicate email/account rejection, non-leader departure, atomic team decisions
T4 - Waivers	Correct user/event association, preview/sign/administrative PDF, removal reflected, private retrieval restrictions
T5 - Timing	Adapter normalization, two-source isolation, shared acquisition, failed switch, idle cleanup, invalid commands, expired socket identity
T6 - Race control	Event-local assignment conflict; pause/resume preserves time; pit exit resets; stopped session requires explicit start; global commands reject kart targeting
T7 - Penalties	Valid values/types; warning threshold; one automatic assessment without clients; deletion does not rearm; next stint can assess
T8 - Results	Valid/mixed/invalid CSV; zero valid rows preserve classification; unauthorized import denied; missing associations/laps shown honestly
T9 - Notifications and expiration	Recipient-only read/delete, reminder deduplication across restarts, provisional 48-hour closure without result publication
T10 - Recovery and usability	Provider/network loss, restart, foreground return, stale indication, account/event switching, accessible status and actionable errors
T11 - Performance	P1–P5 under multi client load
T12 - Access boundaries	Cross-account/team/event attempts, direct private file access, capability-link policy, current-role enforcement, source navigation restrictions

5.4. Acceptance Procedure and Evidence

Run the RASD acceptance examples A1–A10 through the relevant test groups:

- A1–A2 through T2;
- A3 through T3/T12;
- A4 through T5;
- A5/A9 through T6;
- A6 through T7;
- A7 through T8;
- A8 through T4;
- A10 through T10.